CSC1051 • LAB QUIZ 1 Preparation, kuppi session. (1/11/2026)

Java OOP

Sajeer Ahamed

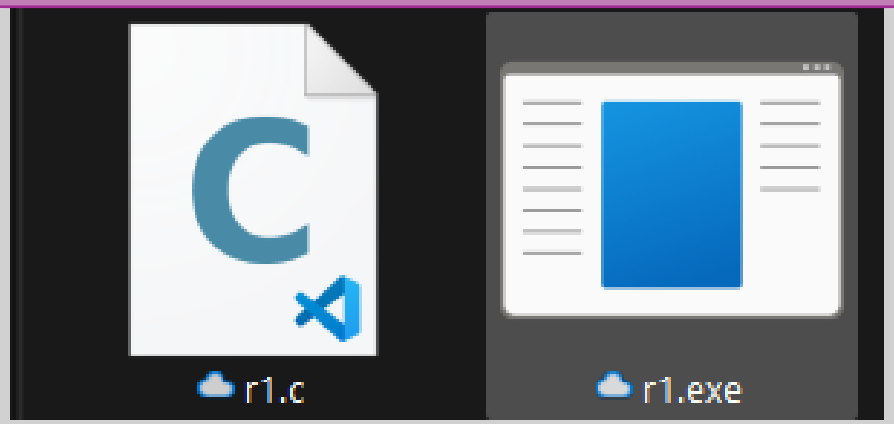
ARCHITECTURE

How Execution Differs: C vs Java

C Model (Native)

Direct native compilation to machine code.

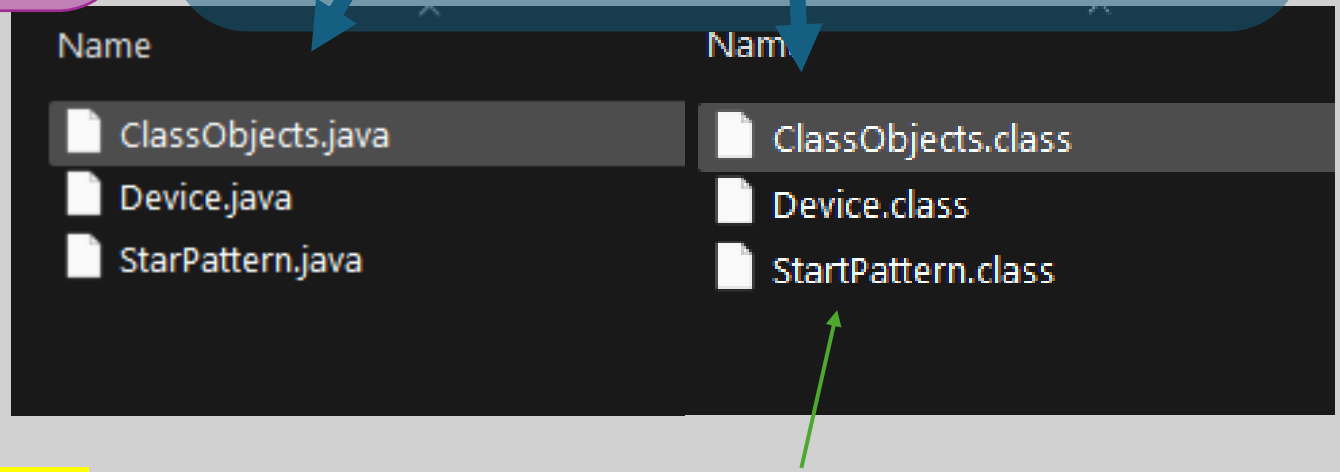
Source (.c) → Machine Code (.exe)



Java Model (JVM)

Two-step compilation and bytecode interpretation.

Source (.java) → Bytecode (.class) → JVM



Key Takeaway: Java achieves platform independence because JVM executes **identical bytecode across systems**.

CODE STRUCTURE

Read more: https://www.geeksforgeeks.org/java/java-source-file-structure/

The Strict Hierarchy of Java

LEVEL 01

Source File

Filename must match public class name.

LEVEL 02

Class Definition

Blueprint holding state and behavior.

LEVEL 03

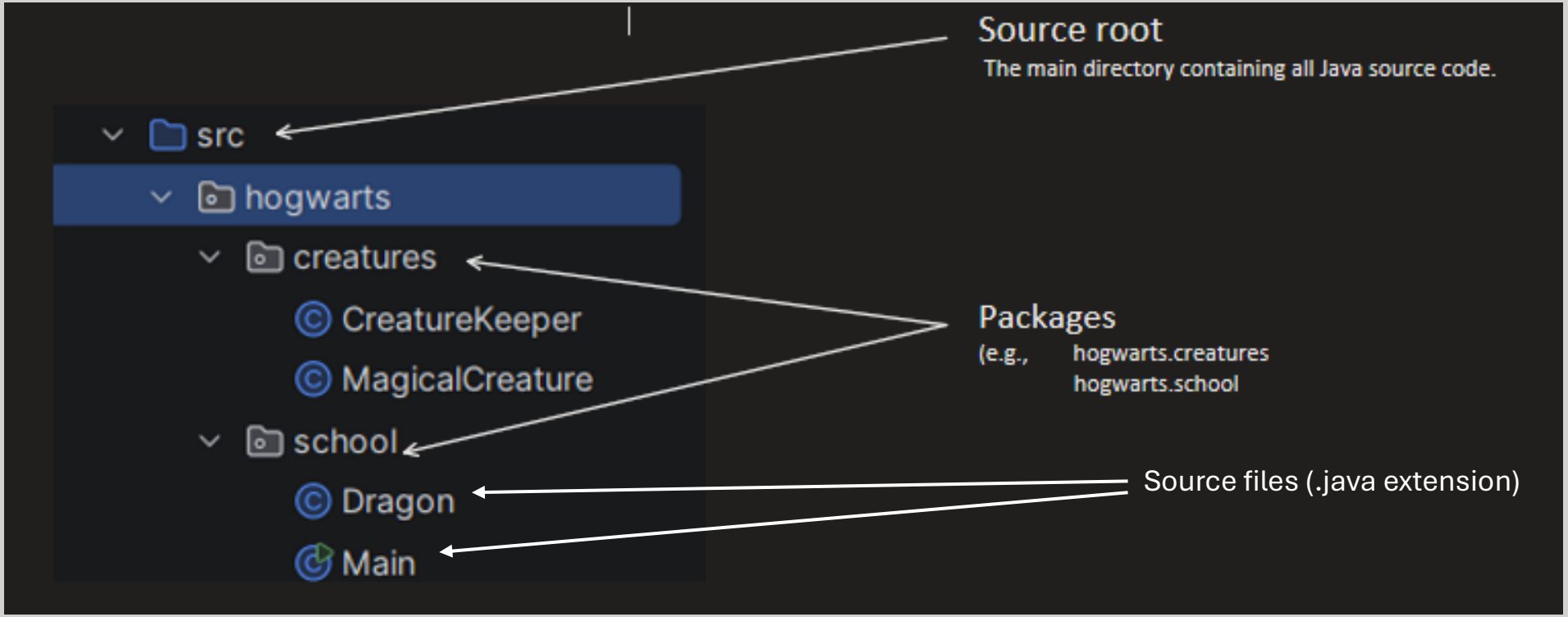
Methods

Functions bound inside classes.

LEVEL 04

Statements

Executable operations with semicolons.




```java
package hogwarts.school;            // Package (Folder Structure)

public class Dragon {               // Level 01 & Level 02: Source File & Class Definition

    public void breatheFire() {     // Level 03: Method
    
        System.out.println("Fire!");// Level 04: Statement ending with ;
    }
}
```

SYNTAX BRIDGE

Translating Core Constructs

Console Output

```c
printf("Age: %d\n", age); // C output

System.out.println("Age: " + age); // java output
```

User Input

```c
int age;// C input
scanf("%d", &age);

Scanner input = new Scanner(System.in); // java input
int age = input.nextInt();
```

Custom Data Types (user defined)

```c
struct Student {     //data bundling in C
    char name[20];
    int mark;
};

class Student {      //data bundling in java
    String name;
    int mark;
}
```

TYPE SYSTEM

Primitives vs Reference Types

8 Primitive Types

byte, short, int, long, float, double, char, boolean

Reference Types

Reference types encompass everything else.

- **Classes** (e.g., `String`, `Scanner`, or custom classes)
- **Arrays** (e.g., `int[]`, `String[]`)
- **Interfaces**, **Enums**, and **Records**

Memory Rule: Primitives hold literal values on the Stack.
Reference variables store memory addresses pointing to objects on the Heap.

Read more : https://medium.com/@abhishekwlpc/primitive-vs-reference-types-in-java-9ecb98ac4c23

https://www.coddykit.com/courses/java/primitive-vs-reference-types-3764884

Class Blueprint vs Object Instance

```java
class Car{
    String color;
    String model;

    void start() {}
}
```

```java
class Main{
    public static void main (String[] args){

        Car myCar = new Car();
        myCar.color = "Red";
        myCar.model = "Corolla";
        myCar.start();

        Car friendsCar = new Car();
        friendsCar.color = "white";
        friendsCar.model = "Swift";
        friendsCar.start();
    }
}
```

Read more : https://www.w3schools.com/java/java_classes.asp

CLASS STRUCTURE

Writing Your First Class: Student.java

```java
public class Student {  3 usages
    // Instance variables / Attributes
    String name;  2 usages
    int mark;  4 usages

    // Constructor
    public Student(String name, int mark) {  1 usage
        this.name = name;
        this.mark = mark;
    }

    void displayGrade(){}  no usages
```

Instance Fields

Store state variables for objects.

Constructor

Initializes object fields upon instantiation.

Instance Methods

Perform behaviors using instance data.

MEMORY ALLOCATION

Dynamic Allocation with 'new' Keyword

```
Student st1 = new Student("Alice", 85);
```

1. Reference Creation

Variable 'st1' created on Stack storing heap reference address.

2. Heap Memory Allocation

'new' allocates space on Heap for fields name and mark.

String in Java

1. String literal (Static Memory):
    - stored in string constant pool.
    - points to duplicate objects mem address
    - immutable

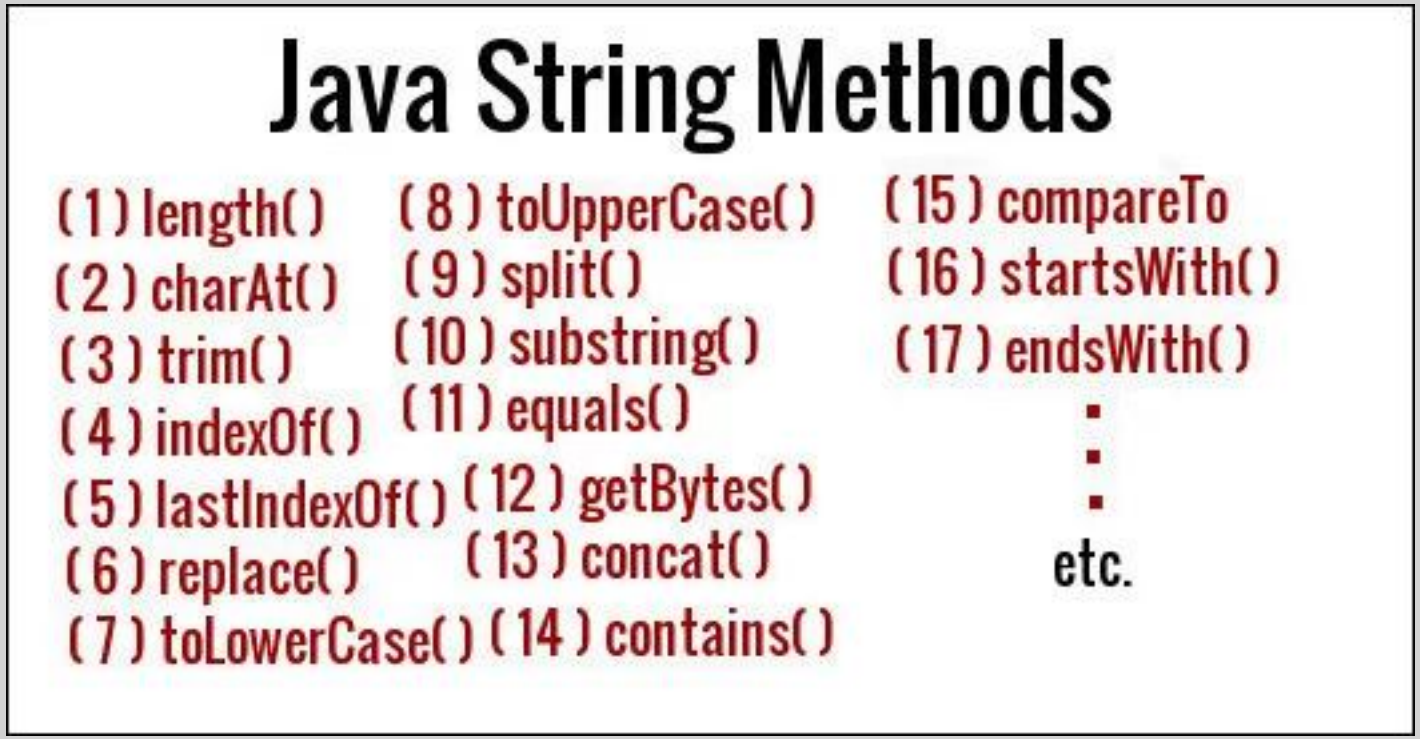
```
int a = 10;
String text = "Hello WOrld"
```

Notice how String is not highlighted unlike int, it means String is an object unlike int which is a primitive data type

2. Using new keyword (Heap Memory)
    - Stored in normal heap
    - Always create new objects

```
String str = new String ("Hellow");
```

String ctd..



Source: https://javatutorial95.blogspot.com/2017/04/string-methods-in-java.html

String ctd..

```java
String str = "Java Programming";
int len = str.length();          // 16
char ch = str.charAt(2);         // 'v'
String sub1 = str.substring( beginIndex: 5);   // "Programming"
String sub2 = str.substring(0,4); // "Java"
```

```java
boolean eq = s1.equals(s2);              // false
boolean eqIgnore = s1.equalsIgnoreCase(s2); // true
int index = "Apple".indexOf("p");        // 1
```

```java
String text = "  Learn Java!  ";
String clean = text.trim();              // "Learn Java!"
String upper = clean.toUpperCase();      // "LEARN JAVA!"
String fixed = clean.replace( target: "!", replacement: "?"); // "Learn Java?"
String con = fixed.concat(s1);
```

Object Arrays in Java

STEP 01: ARRAY CREATION

Student[] list = new Student[3];

Allocates array of 3 null references.

STEP 02: ELEMENT INSTANTIATION

list[0] = new Student("Bob", 72);

Fills null slot with concrete object reference.

Crucial Warning: Unlike C struct arrays, creating a Java object array does NOT instantiate individual objects.
Leaving array slots uninitialized causes NullPointerException.

```
int[] numbers = null;
int length = numbers.length;
int firstNumber = numbers[0];
```

Read more: https://www.geeksforgeeks.org/java/arrays-in-java/

PROGRAM EXECUTION

Complete Driver App Execution Flow

Initialize Scanner

Set up Scanner(System.in) for user input.

Allocate Array

Read size count and declare reference array.

Populate Objects

Loop user inputs and instantiate elements.

Process & Output

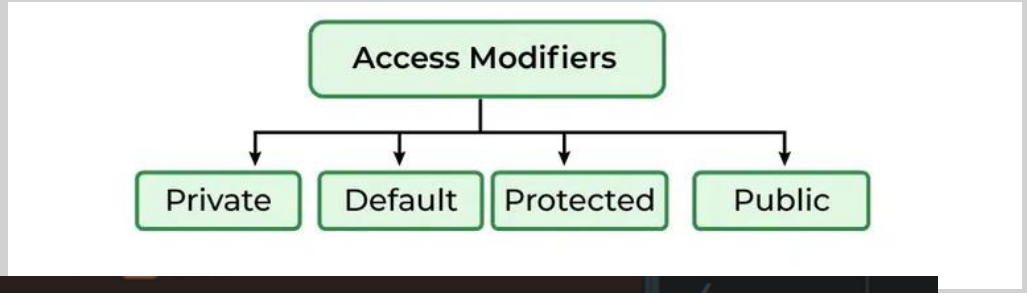
Invoke methods to compute and print results.

```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter number of students: ");
    int count = scanner.nextInt();
    scanner.nextLine();
    Student[] studentList = new Student[count];

    for(int i = 0; i < count; ++i) {
        System.out.println("\nEnter details for Student " + (i + 1) + ":");
        System.out.print("Enter Name: ");
        String name = scanner.nextLine();
        System.out.print("Enter Marks: ");
        int mark = scanner.nextInt();
        scanner.nextLine();
        studentList[i] = new Student(name, mark);
    }
```

Read more: https://www.geeksforgeeks.org/java/access-modifiers-java/

Access modifiers in Java



Same Package (package garage)

Class where attributes are defined (Car.java)
- Top-level class
- different class

Subclass of Car (class Mechanic extends Car)

Unrelated class

Different package /outside package (package showroom)

Subclass of Car (class SportsCar extends Car)

Unrelated class

src
garage
Car.java
Car
Class1
Mechanic
Toyota
showroom
SportsCar
Visitor
.gitignore
acess_mod.iml
External Libraries
Scratches and Consoles

Top-level(public) class

Same .java file, different class

	same package				outside package	
	Same class	same file diff. class	diff. class subclass	diff class not subclass	diff. package subclass	diff. package not subclass (world)
public	✓	✓	✓	✓	✓	✓
protected	✓	✓	✓	✓	✓	✗
Default	✓	✓	✓	✓	✗	✗
private	✓	✗	✗	✗	✗	✗

Question Time!

Ask questions in English Sinhala or Tamil,
Ill try my best to answer

QUIZ PREPARATION

1. String Comparison

Never use == for text values. Always use .equals() method.

2. Scanner Buffer Leak

Call scanner.nextLine() after nextInt() to consume newline.

3. NullPointer in Arrays

Instantiate each element before calling object methods.

4. Class vs File Naming

Source filename must strictly match public class casing.

References

1. *CSC1051: Programming Laboratory semester II — Lecture Slides1 to 4*.
   Department of Statistics and Computer Science - Faculty of Science - University of Peradeniya.
2. https://www.w3schools.com/java/java_intro.asp
3. https://www.coddykit.com/courses/java
4. https://dev.java/learn/language/oop/
5. https://www.geeksforgeeks.org/java/java/